

CallSight AI: AI-Pipeline Analysis and Integration Report

Table of contents

1. What is the AI Pipeline?
2. Where does it live in the system?
3. Full data flow — step by step
4. Pipeline steps in detail
5. File by file walkthrough
6. How models are loaded (loaders.py)
7. PII masking (pii_masker.py)
8. How results are written to the DB (db.py)
9. Error handling & retry logic
10. How the frontend gets the results
11. Key design decisions
12. Requirements & dependencies

Definition

The AI Pipeline is a set of Python scripts that run as **Celery background workers**. It is completely separate from the Django backend — it runs in its own Docker container with its own dependencies (PyTorch, Whisper, Transformers, spaCy). Its sole job is to take a raw audio file and produce structured analysis results.

Key point

The AI Pipeline is NOT a web framework, NOT an API, and NOT a Django app. It is a pure Python worker that listens to Redis for tasks and writes results to PostgreSQL. It has no HTTP server and no endpoints.

What it produces from one audio file

Given a raw audio recording of a call center conversation, the pipeline produces:

Transcript	Full text of the conversation, split by speaker (agent vs client), with timestamps per segment
Sentiment	Sentiment label (positive/neutral/negative) + confidence score for each client segment
Topics	List of detected topics (billing, support, complaint...) with confidence scores
Score	Quality score out of 100 across 6 weighted criteria (accueil, empathie, resolution, langage, conformite, cloture)
Summary	LLM-generated structured summary: main reason for call, actions taken, outcome, and improvement recommendations

System position

The AI Pipeline sits between Redis (the message broker) and PostgreSQL (the database). It never communicates directly with Django or the frontend.

Layer	Technology	Role	Talks to
Frontend	React + Vite	User interface	Django API
Backend	Django DRF	REST API, auth, business logic	Redis, PostgreSQL, S3
Message broker	Redis	Task queue	Django, Celery
AI Pipeline	Celery + Python	ML processing	Redis, PostgreSQL, S3
Database	PostgreSQL	Persistent storage	Django, AI Pipeline
File storage	AWS S3	Audio files	Django, AI Pipeline

The two connections

The AI Pipeline connects to the rest of the system through exactly two channels:

1. **Redis** — Django pushes a task (`call_id`) to Redis after upload. The Celery worker consumes it. This is the *trigger*.
2. **PostgreSQL** — After processing, the pipeline writes all results (transcript, sentiment, topics, score, summary) directly to the shared database using `psycopg2`. Django then reads these results via its ORM when the frontend polls.

#	Action	Layer	Detail
1	User uploads audio	Frontend	Manager drags audio file into UploadModal.jsx
2	Django receives file	Backend	CallViewSet saves metadata to PostgreSQL (status=pending), uploads file to AWS S3
3	Task pushed to Redis	Backend	<code>trigger_analysis.delay(call_id)</code> pushes the task — Django returns 202 immediately
4	Celery picks up task	AI Pipeline	Celery worker consumes task from Redis, calls <code>pipeline.py</code>
5	Download audio	AI Pipeline	<code>s3.py</code> downloads audio from S3 to temp local file
6	Normalize audio	AI Pipeline	<code>audio.py</code> normalizes with <code>ffmpeg</code> , applies VAD, chunks if needed
7	Transcribe	AI Pipeline	<code>transcription.py</code> runs Whisper ASR + <code>pyannote</code> diarization
8	Mask PII	AI Pipeline	<code>pii_masker.py</code> redacts phone numbers, card numbers from transcript
9	Sentiment	AI Pipeline	<code>sentiment.py</code> analyses each client segment with HuggingFace Transformers

10	Topics	AI Pipeline	topics.py runs zero-shot classification with BART
11	Score	AI Pipeline	scoring.py calculates weighted score across 6 criteria
12	Summary	AI Pipeline	summarization.py generates LLM summary + recommendations
13	Write to DB	AI Pipeline	db.py writes all results to PostgreSQL, updates Call status=analyzed
14	Frontend polls	Frontend	useAnalysis.js polls GET /api/calls/{id}/analysis/ every 3 seconds
15	Results displayed	Frontend	CallDetailPage renders transcript, sentiment, score radar, summary

Step	Module	What it does	File
1	transcription.py	Whisper ASR + pyannote diarization — converts audio to timestamped text segments separated by speaker (agent vs client)	transcription.py
2	sentiment.py	HuggingFace Transformers runs sentiment analysis (positive/neutral/negative + confidence) on each client speech segment	sentiment.py
3	topics.py	Zero-shot classification using facebook/bart-large-mnli detects topics: billing, support, complaint, information request, cancellation	topics.py
4	scoring.py	Weighted scoring engine evaluates 6 criteria: accueil (10%), empathie (20%), resolution (30%), langage (20%), conforme (10%), cloture (10%)	scoring.py
5	summarization.py	LLM generates a structured summary: main reason for call, actions taken by agent, call outcome, and specific improvement recommendations	summarization.py

Scoring criteria breakdown

Criteria	Weight	Description
Accueil	10%	Correct greeting and agent self-identification at call start
Empathie	20%	Understanding and acknowledgement of the client's problem
Resolution	30%	Problem was effectively resolved during the call
Langage	20%	Professional tone, appropriate vocabulary, no slang
Conforme	10%	Respect of mandatory scripts and regulatory requirements
Cloture	10%	Correct and complete closing of the call

7. PII masking — pii_masker.py

Before any transcript is written to PostgreSQL, it passes through the PII masker. This module detects and redacts personally identifiable information using regex patterns and spaCy NER (Named Entity Recognition).

PII Type	Example input	Redacted output
Phone numbers	0612345678, +212 6 12 34 56 78	[PHONE]
Credit card numbers	4111 1111 1111 1111	[CARD]
Email addresses	client@example.com	[EMAIL]
Person names (NER)	Je suis Jean Dupont	Je suis [NAME]
National ID numbers	AB123456	[ID]

The AI Pipeline does not use Django's ORM — it uses raw psycopg2 to write results directly to PostgreSQL. This keeps the pipeline completely independent from Django. db.py handles all database writes at the end of the pipeline.

```
import psycopg2, os

def get_connection():

    return psycopg2.connect(

        dbname=os.environ['DB_NAME'],

        user=os.environ['DB_USER'],

        password=os.environ['DB_PASSWORD'],

        host=os.environ['DB_HOST'],

        port=os.environ['DB_PORT']

    )

def write_results(call_id, transcript, sentiment,

    topics, score, summary):

    conn = get_connection()

    cur = conn.cursor()

    # Write transcript

    cur.execute(

        'INSERT INTO analysis_transcript

        (call_id, segments, language_detected)

        VALUES (%s, %s, %s)',

        (call_id, Json(transcript), 'fr')

    )

    # Write sentiment, topics, score, summary...

    # Update call status

    cur.execute(

        'UPDATE calls_call SET status=%s WHERE id=%s',
```



```
('analyzed', call_id)
```

```
)
```

```
conn.commit()
```

```
cur.close()
```

```
conn.close()
```

Celery provides built-in retry logic. If any step in the pipeline fails (network error downloading from S3, model inference error, DB write failure), the task is automatically retried up to 3 times with exponential backoff. If all retries fail, the Call status is set to 'failed' and the PipelineJob records the error message.

```
@app.task(bind=True, max_retries=3,
          default_retry_delay=60) # 60s between retries
def trigger_analysis(self, call_id):
    try:
        # ... run pipeline steps ...

    pass

    except S3DownloadError as exc:

        update_job(call_id, status='failed',

                    error=str(exc))

        raise self.retry(exc=exc,

                           countdown=2 ** self.request.retries)

    except Exception as exc:

        update_job(call_id, status='failed',

                    error=str(exc))

        update_call_status(call_id, 'failed')

        raise
```

Since the pipeline runs asynchronously, the frontend cannot wait for a response. Instead, `useAnalysis.js` polls the Django API every 3 seconds until the analysis is complete.

```
// useAnalysis.js

export function useAnalysis(callId) {

  return useQuery({

    queryKey: ['analysis', callId],

    queryFn: () => api.get(`/calls/${callId}/analysis/`),

    refetchInterval: (data) => {

      // stop polling once analysis is complete

      if (data?.status === 'analyzed') return false

      if (data?.status === 'failed') return false

      return 3000 // poll every 3 seconds

    },

  })

}
```

Call status lifecycle

pending → processing → analyzed (success) or failed (error). The frontend shows a loading spinner while status is 'pending' or 'processing', renders full results when 'analyzed', and shows an error message when 'failed'.

Separate Docker container for AI Pipeline

ML dependencies (PyTorch, Whisper, Transformers) are 5-10GB and require specific CUDA versions. Putting them in the Django container would make it huge and slow to build. A separate container keeps Django fast and allows GPU acceleration only where needed.

Singleton model loading

Loading Whisper large-v3 takes ~2 minutes. With singletons, models load once at worker startup and stay in memory. Every subsequent task runs in 2-3 minutes instead of 4-5 minutes.

Raw psycopg2 instead of Django ORM

The AI Pipeline has no Django installed. Using psycopg2 directly keeps it independent and lightweight. The pipeline only needs to write results — no need for the full ORM.

Sequential pipeline steps

Steps run in sequence (not parallel) because each step depends on the previous one: sentiment needs the transcript, scoring needs sentiment and topics, summary needs everything. Parallelizing would add complexity with no real benefit.

PII masking before DB write

PII is redacted AFTER transcription but BEFORE any DB write. This ensures no sensitive data ever reaches the database, complying with GDPR requirements.

202 Accepted on upload

Django returns 202 immediately after pushing the Celery task. This keeps the API fast — the user doesn't wait for AI processing which can take 2-3 minutes for a 5-minute call.



```
# ai pipeline/requirements.txt
```

```
# Celery
```

```
celery==5.3.4
```

```
redis==5.0.1
```

```
# Audio processing
```

```
ffmpeg-python==0.2.0
```

```
pydub==0.25.1
```

```
# ASR & Diarization
```

```
openai-whisper==20231117
```

```
pyannote.audio==3.1.1
```

```
torch==2.1.0
```

```
torchaudio==2.1.0
```

```
# NLP
```

```
transformers==4.36.0
```

```
spacy==3.7.2
```

```
fr-core-news-sm @ https://...
```

```
# Database
```

```
psycpg2-binary==2.9.9
```

```
# AWS
```

```
boto3==1.34.0
```

```
# Utilities
```

```
python-dotenv==1.0.0
```

```
numpy==1.26.0
```

GPU support

For production on a GPU-enabled VPS, replace `torch==2.1.0` with the CUDA build: `torch==2.1.0+cu118` `--index-url https://download.pytorch.org/whl/cu118`. The Dockerfile handles this automatically based on the `CUDA_AVAILABLE` environment variable.